probability of survival. These sigmoid-transformed weights, along with the final hidden layer value, contribute to the n-dimensional output layer according to eq. (8). See class *PropHazards* in file `nnet_survival.py` in the GitHub repository. The proportional hazards approach is useful for small datasets where one wishes to reduce overfitting by minimizing the number of parameters to optimize. It also makes it easier to interpret the reasons for the model's predictions. This version is very similar to a traditional proportional hazards discrete-time survival model using a complementary log-log link (see (Rodriguez, G., 2016), section 7.5.3: "Discrete Survival and the C-Log-Log Link").

Implementation

We implemented Nnet-survival in the Python language, using the Keras library with Tensorflow backend (code at

`http://github.com/MGensheimer/nnet-survival`). A custom loss function is used which represents the negative log likelihood of the survival model. The output of the neural network is an n-dimensional vector $surv_{pred}$, where n is the number of time intervals. Each element represents the predicted conditional probability of surviving that time interval, or $1-h_j$. An individual's predicted probability of surviving through the end of time interval j is given by eq. (3). An example neural network architecture using the "flexible" version of the discrete time survival model is shown in Fig. 1.

Each individual used to train the model has a known failure/censoring time t and censoring indicator, which are transformed into a vector format for use in the model. Vector $surv_s$ has length n and represents the time intervals the individual has survived through; vector $surv_f$ also has length n and represents the time interval during which failure occurred, if it occurred.

For individuals with failure (uncensored), for time interval j:

$$surv_s(j) = \begin{cases} 1, & \text{if } t \geq t_j \\ 0, & \text{otherwise} \end{cases} \tag{9}$$

$$surv_f(j) = \begin{cases} 1, & \text{if } t_{j-1} \leq t < t_j \\ 0, & \text{otherwise} \end{cases} \tag{10}$$

For censored individuals:

$$surv_s(j) = \begin{cases} 1, & \text{if } t \geq \frac{1}{2}(t_{j-1}+t_j) \\ 0, & \text{otherwise} \end{cases} \tag{11}$$

and

$$surv_f(j) = 0 \tag{12}$$

The log likelihood for each individual is:

$$loglik = \sum_{i=1}^{n} \left(\begin{array}{l} \ln\Big(1 + surv_s(i) \cdot (surv_{pred}(i) - 1)\Big) \\ +\ln\Big(1 - surv_f(i) \cdot surv_{pred}(i)\Big) \end{array} \right) \tag{13}$$

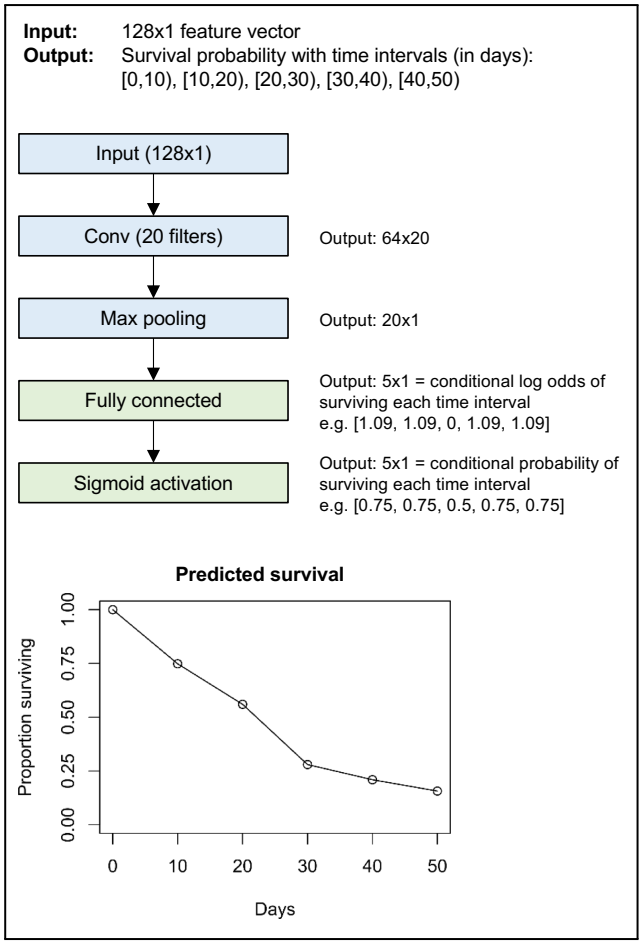


Figure 1. Example neural network architecture and output for one individual. Layers in blue are unique to the example neural network; layers in green are common to all neural networks that use the "flexible" version of our survival model.

which is a restatement of eqs. (5) and (7) to work with the vector encoding of actual and predicted survival.

The loss function is the negative of eq. (13). The loss function is minimized using gradient descent; Keras performs automatic differentiation of the loss function in order to calculate the gradient. In our experiments, using the custom loss function extended running time very slightly compared to standard loss functions such as mean squared error.

The cut-points for the time intervals can be varied according to the specific application. In most of our experiments we have used 15-40 time intervals, spaced out more widely with increasing follow-up time. This ensures that around the same number of survival events fall into each time interval, which may help ensure reliable estimates for all time intervals. Other authors have suggested using at least ten time intervals to avoid bias in the survival estimates (Breslow et al., 1974). In our experiments we have found that the model's performance is fairly robust to choice of specific cut-points.

Performance evaluation: simulated data

We ran several experiments with simulated data to assess correctness of the model. The code is available in `nnet_survival_examples.py` in the GitHub repository.

Simple model with one predictor variable

We first tested a very simple survival model with one binary predictor variable. Five thousand simulated patients were created. Half of the patients had predictor variable value of 0 and were the poor prognosis patients. For this group, survival times were drawn from an exponential distribution with median survival of 200 days. The other half of the patients had predictor variable value of 1 and were the good prognosis patients. Their survival times were drawn from an exponential distribution with median survival of 400 days. For both groups, some patients were censored; censoring time was drawn from an exponential distribution with median value / half-life of 400 days. This survival model used the flexible version of nnet-survival (i.e., non-proportional hazards) with no hidden layers and 39 time intervals spanning the range of 0 to 1780 days.

To evaluate the correctness of this model, we created calibration curves: we plotted and compared actual vs. model-predicted survival curves for the two groups. For each of the two groups, a Kaplan-Meier survival curve was plotted to show actual survival. Then, for each group, a model-predicted survival curve was generated: for each follow-up time point, the average of predicted survival for all patients in that group was calculated and displayed.

Optimal width of time intervals

We investigated whether model performance depended on time interval width. Similarly to the prior example, we simulated a population of 5,000 patients with one binary predictor variable. Survival time distribution was generated using a Weibull distribution, with scale parameter depending on the predictor variable value. Median survival time for the overall population was 182 days. We used the flexible version of nnet-survival to predict survival time. Four options for time intervals were evaluated:

- Uniform intervals with width of 1 year
- Uniform intervals with width of 1 month
- Uniform intervals with width of 1 week
- Increasing width of intervals with increasing follow-up time, with half-life for interval width of 1 year. Specifically, the time interval borders were placed at:

  $\frac{-ln(1-x)\cdot 365}{ln(2)}$ for x in $[0.0, 0.05, 0.10, \ldots, 0.95]$

Discrimination performance was assessed with Harrell's C-index.

Convolutional neural network for MNIST dataset

One area in which neural networks have shown clear superiority to other model types is in analysis of 2D image data, for which convolutional neural networks provide state-of-the-art results. We wished to demonstrate use of a convolutional neural network as part of a survival model. For this, we used the MNIST dataset (Lecun et al., 1998). This dataset includes images of 70,000 handwritten digits with gold-standard labels, divided into a training set of 60,000 digits and a test set of 10,000 digits. We created a simulated scenario in which each image corresponds to one patient, and patients with higher digits tend to have shorter survival. The images could be imagined as an X-ray images of tumors, with higher digits representing larger, more deadly tumors. The goal of the model is to predict survival distribution for each test set patient.

We used only images with digits 0 through 4, leaving 30,596 training set images and 5,139 test set images. Image size was 28x28 pixels. Patients' survival times were drawn from an

exponential distribution with scale parameter depending on digit. The scale parameter (with units of days) was set to

$$\beta = \frac{365 \cdot exp(-0.9 \cdot digit)}{ln(2)} \tag{14}$$

with the probability density function being:

$$f(t; \frac{1}{\beta}) = \frac{1}{\beta} \exp(-\frac{t}{\beta}) \tag{15}$$

Therefore, median survival ranged from 365 days for digit 0 down to 10 days for digit 4. The setup is illustrated in Fig. 2.

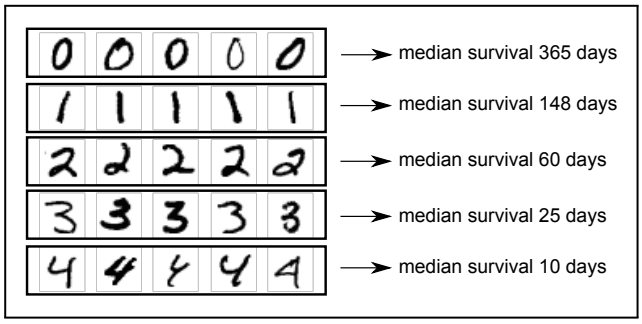


Figure 2. MNIST dataset construction. Images of handwritten digits 0-4 were used as predictor of survival time (one image per patient). Actual survival time was generated from an exponential distribution with scale depending on the digit. Lower digits have longer median survival.

A five-layer neural network architecture was used, with two convolutional layers of kernel size 3x3, followed by a max-pooling layer, a fully connected layer of size 4 neurons, then the output layer. The flexible version of the nnet-survival model was used, so that non-proportional hazards were possible. The Adam optimizer was used. Model performance was evaluated using the C-index to measure discrimination, and calibration curves (actual vs. predicted survival curves) to evaluate calibration. As the Nnet-survival model is flexible and the predicted survival curve for each patient can have a different shape, there is no unique ordering of patients by prognosis (i.e., when comparing two patients, one could have a higher probability of 1-year survival but the other could have a higher probability of 2-year survival). Therefore, to calculate C-index, the model's predicted probability of 1-year survival was used to rank the patients.

Performance evaluation: SUPPORT study (real data)

We evaluated the performance of the Nnet-survival model and other similar models using real patient data. We wished to use a publicly available dataset with time-to-event data on a large number of patients. With a large sample size, we could use data splitting to formally test model performance, and would also be able to evaluate for violations of the proportional hazards assumption of the standard Cox proportional hazards model.

For the real dataset, we chose to use the Study to Understand Prognoses and Preferences for Outcomes and Risks of Treatments (SUPPORT) (Knaus et al., 1995). In this multicenter study, 9,105 hospitalized patients had detailed data recorded including diagnoses, laboratory values, follow-up time and vital status. The dataset is publicly available on the Vanderbilt Biostatistics web site. The task for the survival models was to predict each patient's life expectancy with good discrimination and calibration.